

TEAM ROLES — 4 Members, Zero File Overlap

Rule of thumb: **one folder = one owner**. If a change needs two people's folders touched, it goes through a short function-signature agreement first (see §4), not simultaneous edits.

Member A — Backend / API Lead

Owns: `backend/api/` only.

- Flask app setup, routing, JWT auth (register/login/verify token).
- Request validation for every endpoint.
- Wires together calls to Member B's risk engine and Member D's DB layer — never writes scoring logic or SQL itself.
- Owns the API contract table in `memory.md` — updates it whenever a route's shape changes.

Member B — Risk Engine / ML Lead

Owns: `backend/risk_engine/` only.

- Rule-based scoring (`rules.py`) with human-readable reason strings.
- Feature engineering (`features.py`) and logistic regression model (`ml_model.py` , `train_model.py`).

- Decision assembly: merges rule score + ML probability into ALLOW/STEP_UP/BLOCK (`config.py` holds thresholds).
- Pure functions only — no Flask, no direct DB access — so Member A can call this module without ever opening it.

Member C — Frontend / Dashboard Lead

Owns: `frontend/` only.

- Analyst dashboard: recent transactions, SIM events, alerts, risk scores, agent status.
- OTP verification screen for the customer-facing step-up flow.
- Consumes `GET /api/dashboard/summary` and `POST /api/otp/verify` — never edits backend code; if a response shape needs to change, raises it with Member A first and it lands in the `memory.md` API contract table before either side codes against it.

Member D — Data / Integration / QA Lead

Owns: `backend/db/`, `simulators/`, `notifications/`, and top-level test/CI config.

- Database schema and persistence layer (`models.py`, `schema.sql`, `seed.py`).
- Synthetic traffic generator and demo/monitoring agents (heartbeat status).
- SMS/email notification module (Twilio/SMTP integration, using env vars from `TECH_STACK.md` — never hardcodes a key).

- Owns `tests/` folder structure and a basic CI workflow; writes integration tests that hit Member A's API end-to-end.

3. Shared, read-only files (anyone can propose a PR, no one edits solo)

`PRD.md`, `ARCHITECTURE.md`, `memory.md`, `README.md`,
`.env.example`, `requirements.txt` —

changes to these go through a PR the other three can see, since they affect everyone.

4. How to avoid merge conflicts in practice

- Git branch naming: `a/feature-name`, `b/feature-name`,
`c/feature-name`, `d/feature-name`.
- Before writing code that calls into someone else's module, agree the function signature or endpoint shape in a 2-line Slack/WhatsApp message and drop it into `memory.md`'s API contract table — then both people can build against the *agreed shape* without waiting on each other's actual implementation.
- Nobody pushes directly to `main`. Every merge is a PR reviewed by at least one other member.
- If two people genuinely need to touch the same file (rare, e.g. `requirements.txt`), whoever gets there first pulls, edits, pushes immediately — don't hold a shared file open across a work session.

5. Suggested week-by-week split (adjust to your timeline)

Week	A (API)	B (Risk Engine)	C (Frontend)	D (Data/Sim/QA)
1	Flask skeleton, auth	Rule engine v1 + stub ML	Dashboard shell, mock data	Schema + seed data
2	Risk/OTP routes wired to real B+D	Train logistic regression on synthetic data	Wire dashboard to real API	Traffic simulator + notifications
3	Dashboard summary endpoint, hardening	Tune thresholds from live synthetic traffic	OTP flow, polish UI	Agent heartbeat, integration tests
4	Bug fixes, docs	Explainability polish (reason strings)	Final UI pass	End-to-end demo script, CI